BayesianEstimationTool Manual

BET fits models with population-wide parameters and individual-specific latent variables. This guide covers the actual model contracts, inference, array shapes, masking, and JAX modeling patterns.

1. Input and output contracts

`Estimator(data, ModelFn, localZNames=None, GlobalPriorFn=None, lkjConcentration=2.0, rngSeed=None)` takes a dictionary of arrays. The first axis of each data array indexes individuals; all arrays have the same number of individuals. Remaining axes can represent times, features, or spatial coordinates.

`ModelFn(globalParams, localZs, data)` receives dictionaries of global parameters, one individual's local latent values, and one individual's data. It returns a dictionary of model outputs. BET vectorizes this function across individuals and posterior draws; avoid Python side effects and conversions of traced JAX values to NumPy.

`GlobalPriorFn(dataShapes)` returns a dictionary of parameter names mapped to NumPyro distribution objects. Each shape in `dataShapes` excludes the individual axis. The global parameters are shared by the training population.

`TrainLikelihoodFn(globalParams, localZs, data, modelOut)` and `TestLikelihoodFn` each return a dictionary of site names mapped to `(observed_value, distribution)` tuples. A site may represent one scalar or a vector of observations; observation, distribution, and mask shapes must agree.

`ScoreFn(modelOut, data)` returns a scalar or array for one individual. BET vectorizes it over posterior samples and individuals.

2. Minimal working model

```
import jax.numpy as jnp
import numpyro.distributions as dist
from BayesianEstimationTool import Estimator, Svi

data = {"x": jnp.arange(8.0), "y": jnp.arange(8.0) + 0.2}

def global_prior(shapes):
    return {"slope": dist.Normal(1.0, 0.5)}

def model_fn(global_params, local_zs, data):
    return {"prediction": global_params["slope"] * data["x"]}

def likelihood(global_params, local_zs, data, model_out):
    return {"y": (data["y"], dist.Normal(model_out["prediction"], 0.5))}

def score(model_out, data):
    return jnp.abs(model_out["prediction"] - data["y"])

est = Estimator(data, model_fn, GlobalPriorFn=global_prior, rngSeed=1)
est.Train([0, 1, 2, 3, 4, 5], likelihood,
          Svi(numSteps=200, numSamples=100, progressBar=False))
est.Test([6, 7], nLocalZSamples=10)
train_scores = est.ScoreTrainSamples(score)
```

BET supplies one individual's data to `ModelFn`; do not index a patient axis within the function.

3. Hierarchical local variables

`localZNames=["growth", "response"]` introduces standardized normal latent variables for each individual. With multiple local variables, BET uses an LKJ prior for their Cholesky correlation structure, controlled by `lkjConcentration`. The user model converts standardized values into meaningful parameters.

```
def model_fn(global_params, local_zs, data):
    growth = jnp.exp(global_params["log_growth"] + 0.3 * local_zs["growth"])
```

```
    return {"prediction": data["baseline"] * jnp.exp(growth * data["time"])}
```

BET does not automatically infer an independent population scale or mean for each local variable; define these transformations explicitly when required.

4. Training, testing, and score dimensions

`Train(trainDataIdxs, TrainLikelihoodFn, Method, printSummary=False, rngSeed=None, saveLocalZs=True, saveModelOut=False)` estimates parameters from the selected individuals using a `Nuts(...)` or `Svi(...)` method.

`Test(testDataIdxs, TestLikelihoodFn=None, Method=None, nGlobalSamples=None, nLocalZSamples=2000, printSummary=False, rngSeed=None, saveModelOut=False)` supports prior prediction for new individuals when no test likelihood is supplied, or posterior conditioning when a test likelihood and Method are supplied. `nGlobalSamples=None` uses posterior medians; a positive value samples stored training draws. `nLocalZSamples` applies to the no-likelihood mode.

`ScoreTrainSamples(ScoreFn, nSamples=None, rngSeed=None)` returns `[posterior_draw, train_individual, ...score_shape]`.

`ScoreTestSamples(ScoreFn, nSamples=None, rngSeed=None)` returns `[global_draw, local_draw, test_individual, ...score_shape]`. For local-free models, save model outputs if scoring a test run.

5. Variable-length sequences and masking

Pad individual time series to a common length `T`, storing `time`, `y`, and a boolean `valid` mask as `[N, T]` arrays. Padded entries are storage placeholders, not observations.

```
data = {
    "time": jnp.array([[0., 1., 2.], [0., 3., 0.]]),
    "y": jnp.array([[1.0, 1.2, 1.4], [0.9, 1.1, 0.0]]),
    "valid": jnp.array([[True, True, True], [True, True, False]]),
}
```

Use a masked NumPyro distribution and replace invalid observed values with finite, safe values before log-probability evaluation:

```
def likelihood(global_params, local_zs, data, model_out):
    valid = data["valid"]
    pred = model_out["prediction"]
    safe_y = jnp.where(valid, data["y"], pred)
    return {"y": (safe_y, dist.Normal(pred, 0.2).mask(valid))}
```

The model must still produce finite predictions at padded positions. A zero mask does not guarantee safety if an intermediate expression already contains `NaN` or infinity. Do not multiply a divergent log-probability by zero and assume its gradient is safe.

To hide additional data during testing, store a separate observation mask, such as `test_observed = valid & (time <= cutoff)`. The test likelihood uses this mask; scoring may use the original validity mask to evaluate held-out observations. Do not leak hidden measurements into the model inputs or preprocessing.

6. JAX differential equation solvers

The model function may call JAX-compatible ODE solvers, including `jax.experimental.ode.odeint` or Diffrax. Keep solver inputs JAX arrays, use differentiable operations, and return outputs at the observation times. Diffrax is an optional external dependency, not part of BET's installation requirements.

```
from jax.experimental.ode import odeint

def rhs(state, time, rate):
```

```
    return rate * state

def model_fn(global_params, local_zs, data):
    rate = jnp.exp(global_params["log_rate"] + local_zs["rate"])
    times = data["time"]  # sorted, finite, compatible with odeint
    trajectory = odeint(rhs, data["initial"], times, rate)
    return {"prediction": trajectory}
```

With padded time grids, ensure solver times satisfy the solver's ordering requirements. A common approach is to use a shared regular solver grid and interpolate to each individual's observation times. Beware duplicate or unsorted padded times.

7. Numerical stability and diagnostics

Scale observations and model outputs to reasonable ranges, and use likelihood scales reflecting measurement noise rather than arbitrary tiny numbers. Extremely large residuals or near-zero noise can produce unstable log-probabilities and gradients. Replace missing observations with safe values before masking, and ensure model outputs remain finite even at padded positions. Use `jax.debug.print`, `jax.numpy.isfinite`, and small synthetic examples to isolate shape or gradient errors.

For multiple individuals, check each axis and broadcasting rule explicitly. Test the model and likelihood on one individual before fitting the population. Use posterior predictive checks; a successful NUTS or SVI run does not guarantee a scientifically adequate model.

8. Persistence and custom methods

`est.Save("fit.npz", saveAllData=True)` saves inference state and optionally data. Restore with `Estimator.Load("fit.npz", ModelFn=model_fn, GlobalPriorFn=global_prior)` when data was saved, or provide `data=` when it was not. Supply the original model and prior functions again; they are not serialized as executable code.

Custom inference methods must implement `_Fit(ModelFn, rngKey, samplePrefixes)` and `_SaveArgs()`. Run `ValidateMethod(method)` before use. See `METHOD_AUTHORING.md` for the minimum viable implementation and optional features.

9. Test strategy

The regression suite exercises NUTS, SVI, scoring, validation, and Save/Load. Scientific validation additionally requires synthetic hierarchical parameter recovery, population-to-new-individual transfer with varying observation counts, and numerical equivalence after persistence. See `TESTING.md` for the test invocation.